

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. שנים עשר קשיים, מסודרים לפי הסדר שבו הם קורים בפועל — לא לפי סדר החלקים בברף. הכלי שפותר את רובם הוא DevTools, לשונית Network, מסונן ל-Fetch/XHR. שם רואים אם בקשה יצאה בכלל, כמה יצאו, מה הסטטוס שחזר, ומה היה בגוף. פתח אותה עכשיו והשאר אותה פתוחה. ולשונית Elements, על #api-panel — ה- data-state שם אומר לך מה היישום חושב שקורה, וזה כמעט תמיד השאלה.

א · "לא קורה כלום כשאני מקליד"

דחיפה. פתח את Network וסנן ל-Fetch/XHR. הקלד. יצאה בקשה? התשובה הזאת מפצלת את כל הבעיה לשתיים, ולוקח עשר שניות לקבל אותה.

אסטרטגיה. אם לא יצאה בקשה — הבעיה לפני הרשת: או שהמאזין על #api-query לא נרשם — בדוק שהוא בתוך wire, ושקוראים ל-wire בכלל; או ש-scheduleSearch יוצאת מוקדם, או שהטיימר נוצר ומיד נדרס. אם יצאה בקשה — הבעיה אחריה: הסתכל בסטטוס ובלשונית Response.

כמעט-פתרון. אם בקונסולה כתוב Uncaught (in promise) — הפונקציה שלך async, היא זרקה, ואיש לא תפס. runSearch נקראת מטיימר, ואף אחד לא מחכה לה: כל throw בתוכה נעלם לתוך הקונסולה במקום להגיע למסך.

ב · "התשובה חוזרת אבל אני מקבל שגיאה"

דחיפה. בלשונית Network, לחץ על הבקשה ותסתכל ב-Response. מה בדיוק חזר? העתק את זה לקונסולה והרץ עליו JSON.parse ידנית.

אסטרטגיה. שלוש אפשרויות, ובכולן res.ok הוא true: הגוף אינו JSON (עמוד התחברות של Wi-Fi), הגוף JSON אבל לא במבנה שציפית, או שהמנרמל שלך מניח מפתח שלא קיים.

כמעט-פתרון. תסתכל על שורת ה-map שלך. אם היא payload.results.map(...) והתשובה היא {} — זה מגיע ל-catch שלך ונראה כמו תקלת רשת. הרבה APIs משמיטים את מפתח התוצאות כשלא נמצא כלום. תבדוק את שלך בשורת הכתובת.

ג · "404 מציג לי תוצאות ריקות במקום שגיאה"

דחיפה. fetch לא זרק. הוא לא זורק על 404. תבדוק מה res.ok מחזיר.

אסטרטגיה. ההבטחה של fetch נדחית במקרה אחד בלבד: הבקשה לא הושלמה — DNS, אין נתיב, CORS, ביטול. 404 ו-500 הן תשובות מוצלחות של HTTP: השרת קיבל, עיבד וענה. הוא ענה "לא".

כמעט-פתרון. שורה אחת, מיד אחרי ה-await fetch ולפני ה-json():

```
if (!res.ok) throw new ApiError('status', `... (${res.status})`, `HTTP ${res.status}`);
```

זה הכול. השורה הזאת היא רבע מהשיעור, והיא לא קיימת ברוב הקוד שנכתב על ידי מודל.

ד · הפאנל מהבהב באדום בזמן שאני מקליד

דחיפה. אתה מבטל את הבקשה הקודמת בכל הקשה, נכון? אז מה קורה ל-promise המבוטלת?

אסטרטגיה. ביטול מגיע כ-**דחייה**. `controller.abort()` גורם ל-`fetch` להידחות עם `DOMException` בשם `AbortError` – והוא נוחת ב-`catch` שלך, בדיוק כמו כישלון אמיתי. ה-`catch` שלך מכניס אותו ל-`state` כשגיאה, ו-`render` מצייר פאנל אדום. פעם אחת לכל תו.

כמעט-פתרון. השורה הראשונה ב-`catch`, לפני כל דבר אחר:

```
if (error.name === 'AbortError') return;
```

`return`, ולא `throw`: הקורא שלך הוא טיימר, ו-promise שנדחה ואיש לא ממתין לו הוא `unhandled rejection`. ובדוק **לפי שם** ולא **לפי מחלקה** – `TimeoutError` הוא גם `DOMException`, והוא כן שגיאה.

ה · ייש לי בקשה לכל תו

דחיפה. ספור אותן ב-`Network`. הקלד מילה בת שש אותיות. כמה שורות?

אסטרטגיה. צריך **השהיה**: אחרי כל הקשה, מבטלים את הטיימר הקודם ומתזמנים חדש ל-300 מילישניות. אם המשתמש הקליד עוד אות בינתיים – הטיימר הקודם נהרג ולא ירה מעולם.

כמעט-פתרון. אם יש לך `setTimeout` ואתה עדיין רואה שש בקשות – **חסרה לך שורה אחת**, והיא `clearTimeout(timer)` לפני ה-`setTimeout`. בלעדיה כל הקשה מתזמנת טיימר משלה ואף אחד לא מבטל, אז שש הקשות הן שש בקשות שרק קורות 300 מילישניות מאוחר יותר.

ו · "לפעמים מוצגות תוצאות של מילה אחרת"

דחיפה. תקליד מילה ארוכה לאט, ותסתכל על מה שמוצג אחרי כל אות. זה **תמיד** תואם למה שבתיבה?

אסטרטגיה. שתי בקשות פתוחות בו זמנית, והתשובה שנוחתת **אחרונה** היא זו שנשארת. חיפוש קצר מחזיר יותר תוצאות ולוקח יותר זמן, אז זו לרוב תשובת החיפוש **הישן**. זה לא נדיר, זו ברירת המחדל.

כמעט-פתרון. שני תיקונים ושניהם נחוצים:

```
controller?.abort(); // the previous one is not wanted
controller = new AbortController();
const ticket = ++seq;
const results = await search(query, { signal: controller.signal });
if (ticket !== seq) return; // a newer search has started
```

הביטול עוצר את **הרשת**. המספר עוצר את **הכתיבה**. הם לא אותו דבר.

ז · "מצב הטעינה לא מופיע"

דחיפה. איפה בדיוק בקוד אתה כותב `status: 'loading'` לפני ה- `await` או אחריו?

אסטרטגיה. הקוד שאחרי `await` לא רץ בזמן ההמתנה. הוא רץ כשההמתנה נגמרה. אם קבעת "טוען" שם, המצב נכון בדיוק ברגע שבו הוא כבר לא נכון.

כמעט-פתרון. `setState({ status: 'loading', error: null })` הוא הדבר הראשון ב- `runSearch`, לפני שנוצר `AbortController` ולפני שנשלחת בקשה. ה- `error: null` שם לא לקישוט: אם תשאיר שם את הכישלון הקודם, תקבל פאנל שהוא גם טוען וגם אדום.

ולבדיקה: `DevTools > Network > Throttling > Slow 3G`. בלי זה הכול מהיר מדי מכדי לראות מצב טעינה, וזה בדיוק למה הוא נשכח.

ח · "ספינר ולא נמצא כלום" מוצגים ביחד"

דחיפה. כמה שדות ב-`state` מחליטים איזה אזור מוצג? אם התשובה גדולה מאחד — מצאת.

אסטרטגיה. `isLoading` ו- `isEmpty` כשני בוליאנים הם ארבעה צירופים, ורק שלושה מהם אומרים משהו. הרביעי מגיע כשענף אחד קובע את שלו ושוכח לנקות את השני — וזה תמיד קורה, כי הענף הרביעי נוסף חודשיים אחרי השלושה הראשונים.

כמעט-פתרון. שדה אחד, חמישה ערכים, וגזירה אחת שמחשבת את התצוגה:

```
if (status === 'loading') return 'loading';
if (status === 'error') return 'error';
if (status === 'idle') return 'idle';
return results.length === 0 ? 'empty' : 'results';
```

ואז ב- `render` — `const view = selectSearchView(state)` פעם אחת, וארבע שורות `hidden` שכולן משוות ל- `view`. אין דרך להגיע לשני אזורים.

ט · "לא נמצא כלום" כתוב לי כבר בטעינת העמוד"

דחיפה. בטעינה, `results` הוא מערך ריק. מה הגזירה שלך מחזירה עבורו?

אסטרטגיה. `results.length === 0` נכון גם כשעוד לא שאלת כלום. `idle` ו- `empty` הם שני מצבים שונים לגמרי: אחד שולח את המשתמש לחיבת החיפוש, והשני אומר לו שחיבת החיפוש כבר אכזבה אותו.

כמעט-פתרון. בדיקת ה- `status === 'idle'` צריכה לבוא לפני בדיקת האורך. ראה את הקטע בקושי ח: הסדר שם הוא התשובה.

י · "TypeError: Failed to fetch ואין שום פרט"

דחיפה. הכתובת עובדת אם אתה מדביק אותה בשורת הכתובת? אם כן — זה `CORS`.

אסטרטגיה. פתח את `Network` והסתכל על השורה של הבקשה. היא שם, יש לה סטטוס, ויש לה גודל. הבקשה יצאה, השרת עיבד וענה, והדפדפן סירב למסור לך את התשובה כי היא לא נשאה כותרת שאומרת שמותר לך לקרוא אותה. הסיבה מודפסת בקונסולה על ידי הדפדפן, ואין שום דרך לקרוא אותה מ-`JavaScript` — בכוונה.

כמעט-פתרון. אין תיקון בקוד שלך, ולא יהיה. `mode: 'no-cors'` מפסיק את השגיאה ונותן לך תשובה **אטומה**: סטטוס 0, בלי גוף, בלי כלום. `try/catch` תופס את ה-`TypeError` ולא מביא נתונים. **בחר API שכן שולח את הכותרת** — יש רשימה בסוף הבריק — ותעד את זה שניסית בהערה. זה חלק ה-2, וזה שווה נקודות.

יא • `fetch` לקובץ המקומי נכשל

דחיפה. איך פתחת את העמוד? אם בכתובת `file:///` — זה זה.

אסטרטגיה. ל-`file://` אין `origin` אמיתי, ולכן `fetch` לכל קובץ ממנו הוא בקשה חוצת-מקורות אל "null", והדפדפן חוסם אותה. **זה הדבר היחיד ביישום שנשבר ככה** — הרשימה, הטופס, האחסון וכל השאר עובדים מצוין ב-`file://`, ולכן זה נראה כמו באג בקוד שלך.

כמעט-פתרון. `Live Server`. תמיד. זה הכלל משבוע 1, וזו הפעם הראשונה שהוא מפוצץ משהו. אם הוא כן רץ ואתה עדיין מקבל 404 — הנתיב. `fetch('data/catalogue.json')` הוא יחסי ל-**עמוד**, לא לקובץ ה-`JS`.

יב • "הפריט מהקטלוג לא נשמר"

דחיפה. תבדוק ב-`Application > Local Storage` אם הוא בפנים. אם כן — הבעיה בקריאה, ואם לא — בכתיבה.

אסטרטגיה. אתה **לא אמור** לכתוב שום קוד שמירה השבוע. המטפל קורא ל-`setState`, `persist` רשומה על `setState` מ-`app.js` משבוע 10, וזהו. אם זה לא קורה — או ש-`app.js` שלך איבד את `subscribe(persist)`, או שהמטפל שלך לא עובר דרך `setState` בכלל.

כמעט-פתרון. אם המטפל שלך עושה `state.items.push(item)` — זה הבאג, והוא של שבוע 9. **משנה את המערך הקיים**, אז `state.items === lastSaved` נשאר נכון ושכבת האחסון מדלגת על הכתיבה. תבנה מערך חדש: `setState({ items: [...state.items, item] })`.

כשהכול נשבר בבת אחת

קרה משהו לרשת של המעבדה? `node mock-api/server.mjs`, ומכוונים את `ENDPOINT` אליו. יש שם גם `?fail=404`, `?fail=500`, `delay=8000`? ו-`hang=1`? — כלומר כל התקלות שאתה צריך לבדוק, בפקודה, בלי לחכות לתקלה של מישהו אחר. קרא את `mock-api/README-he.md`.